Day 5 handout

Day 5 Handout: Data Manipulation and Analysis with Pandas and SciPy

PhD Course in Integrative Neurosciences - Introduction to Scientific Programming

---

Table of Contents

1. Introduction to the Data Analysis Pipeline
2. Pandas Fundamentals
3. Reading Data Files
4. Data Exploration
5. Data Selection and Indexing
6. Data Cleaning
7. Data Transformation
8. Grouping and Aggregation
9. Reshaping and Combining Data
10. Visualization with Pandas
11. Working with Datetime Data
12. Signal Processing Basics
13. The SciPy Library
14. Statistical Testing
15. Distribution Fitting
16. Correlation Analysis
17. Best Practices and Workflow

---

1. Introduction to the Data Analysis Pipeline

What is Data Analysis?

Data analysis is the systematic process of inspecting, cleaning, transforming, and modeling data to discover useful information, draw conclusions, and support decision-making. In neuroscience research, this process is fundamental to extracting meaningful insights from experimental data.

The Scientific Data Analysis Pipeline

A typical data analysis workflow consists of several interconnected stages:

1. **Data Acquisition** - Collecting raw data from experiments, databases, or simulations
2. **Data Loading** - Reading data into Python from various file formats (CSV, Excel, etc.)
3. **Data Exploration** - Understanding data structure, types, distributions, and potential issues
4. **Data Cleaning** - Handling missing values, removing duplicates, correcting errors
5. **Data Transformation** - Creating new variables, aggregating, reshaping data
6. **Data Analysis** - Statistical testing, modeling, pattern discovery
7. **Visualization** - Creating informative plots to communicate findings
8. **Interpretation** - Drawing conclusions and relating findings back to research questions

Why Pandas and SciPy?

Pandas is the cornerstone library for data manipulation in Python. It provides:

- High-performance data structures (Series and DataFrame)
- Tools for reading and writing data across multiple formats
- Integrated handling of missing data
- Flexible reshaping and pivoting of datasets
- Intelligent label-based slicing and indexing
- Powerful groupby functionality
- Built-in visualization capabilities

SciPy complements pandas by providing:

- Statistical functions and hypothesis testing
- Distribution fitting and analysis
- Signal processing capabilities
- Optimization algorithms
- Linear algebra operations
- Scientific computing utilities

Together, pandas and SciPy form a powerful ecosystem for scientific data analysis that rivals specialized statistical software packages while offering the flexibility and power of Python programming.

The Pandas + SciPy Ecosystem

Pandas and SciPy don't work in isolation. They're part of a broader scientific Python ecosystem:

- **NumPy** - Underlying array operations and numerical computing
- **Matplotlib** - Core visualization library
- **SciPy** - Scientific computing and advanced statistics
- **Pandas** - Data structures and analysis tools
- **Seaborn** - Statistical data visualization (next lecture!)
- **Jupyter** - Interactive development environment

---

2. Pandas Fundamentals

Core Data Structures

Pandas provides two primary data structures:

Series: One-Dimensional Labeled Array

A Series is a one-dimensional array-like object containing a sequence of values and an associated array of labels (the index).

Key characteristics:

- Homogeneous data (all elements have the same data type)
- Size-immutable (length cannot change after creation)
- Values are mutable (can be changed)
- Labeled index for each element

Creating a Series:

```
import pandas as pd
import numpy as np

# From a list
reaction_times = pd.Series([245, 312, 289, 276, 301])

# With custom index
reaction_times = pd.Series(
    [245, 312, 289, 276, 301],
    index=['trial_1', 'trial_2', 'trial_3', 'trial_4', 'trial_5']
)

# From a dictionary (keys become index)
gene_expression = pd.Series({
    'BDNF': 1.2,
    'GRIN2A': 0.8,
    'DRD2': 1.5,
    'HTR2A': 0.9
})
```

Accessing Series elements:

```
# By position (like a list)
reaction_times[0]  # First element
reaction_times[:3]  # First three elements

# By label (using the index)
reaction_times['trial_1']
gene_expression['BDNF']
```

```
# Boolean indexing
reaction_times[reaction_times > 280]
```

DataFrame: Two-Dimensional Labeled Data Structure

A DataFrame is a two-dimensional labeled data structure with columns of potentially different types. Think of it as a spreadsheet or SQL table in Python.

Key characteristics:

- Tabular data with rows and columns
- Columns can have different data types
- Size-mutable (rows and columns can be added/deleted)
- Labeled axes (rows and columns)
- Can be thought of as a dictionary of Series

Creating a DataFrame:

```
# From a dictionary of lists
data = {
    'subject_id': [1, 2, 3, 4, 5],
    'condition': ['control', 'drug', 'control', 'drug', 'control'],
    'reaction_time': [245, 312, 289, 276, 301],
    'accuracy': [0.95, 0.88, 0.92, 0.90, 0.94]
}
df = pd.DataFrame(data)

# From a list of dictionaries
data = [
    {'subject': 1, 'rt': 245, 'accuracy': 0.95},
    {'subject': 2, 'rt': 312, 'accuracy': 0.88},
    {'subject': 3, 'rt': 289, 'accuracy': 0.92}
]
df = pd.DataFrame(data)

# From NumPy array with column names
data = np.random.randn(100, 4)
df = pd.DataFrame(
    data,
    columns=['neuron_1', 'neuron_2', 'neuron_3', 'neuron_4']
)
```

DataFrame attributes:

```
df.shape        # (rows, columns)
df.columns      # Column names
df.index        # Row labels
df.dtypes       # Data types of each column
df.values       # Underlying NumPy array
df.size         # Total number of elements
```

Pandas and Jupyter Notebooks

Pandas is designed to work seamlessly with Jupyter notebooks, providing:

1. **Rich HTML display** - DataFrames render as formatted tables
2. **Interactive exploration** - Easy to test operations and see results
3. **Inline plotting** - Visualizations appear directly below code cells
4. **Tab completion** - Discover methods and attributes easily
5. **Documentation access** - Use `?` or `??` to view help

Jupyter-specific pandas settings:

```
# Display more rows
pd.set_option('display.max_rows', 100)

# Display more columns
pd.set_option('display.max_columns', 50)

# Wider display
pd.set_option('display.width', 1000)

# More decimal places
pd.set_option('display.precision', 4)

# Reset to defaults
pd.reset_option('all')
```

---

3. Reading Data Files

> **About the examples.** Unless a snippet creates its own data, `df` is the DataFrame from Exercise 1.1 (columns `subject_id`, `condition`, `trial`, `reaction_time`, `accuracy`, `session`; see `day05_exercises.md`). Other column names that appear in the examples (`age`, `iq`, `gene_name`, ...) and file names such as `data.csv` are placeholders for your own data.

Reading CSV Files

CSV (Comma-Separated Values) is the most common format for tabular data. Pandas provides `read_csv()` with extensive options for handling various CSV formats.

Basic CSV reading:

```
df = pd.read_csv('behavioral_data.csv')
```

Common parameters:

```
df = pd.read_csv(
    'data.csv',
    sep=',',              # Delimiter (default: comma)
    header=0,             # Row to use as column names (default: 0)
```

```
    index_col=0,          # Column to use as row labels
    usecols=['col1', 'col2'],  # Read only specific columns
    nrows=1000,           # Read only first N rows
    skiprows=[0, 2, 5],   # Skip specific rows
    na_values=['NA', '?', '-'],  # Additional strings to recognize as
NaN
    encoding='utf-8',     # Character encoding
    parse_dates=['date_col'],  # Parse dates from strings
    dtype={'age': int}    # Specify column data types
)
```

Reading tab-separated files (TSV):

```
df = pd.read_csv('data.tsv', sep='\t')  # Tab-separated
```

Handling files without headers:

```
df = pd.read_csv(
    'data.csv',
    header=None,
    names=['subject', 'trial', 'rt', 'accuracy']
)
```

Reading from URLs:

```
url = 'https://example.com/data.csv'
df = pd.read_csv(url)
```

Reading Excel Files

Excel files (.xlsx, .xls) are common in research settings. Pandas uses the `openpyxl` library for modern Excel formats.

Basic Excel reading:

```
df = pd.read_excel('experimental_data.xlsx')
```

Reading specific sheets:

```
# By sheet name
df = pd.read_excel('data.xlsx', sheet_name='Results')

# By sheet index (0-based)
df = pd.read_excel('data.xlsx', sheet_name=0)

# Read multiple sheets (returns dictionary of DataFrames)
dfs = pd.read_excel('data.xlsx', sheet_name=['Sheet1', 'Sheet2'])
df1 = dfs['Sheet1']
df2 = dfs['Sheet2']

# Read all sheets
all_sheets = pd.read_excel('data.xlsx', sheet_name=None)
```

Excel-specific parameters:

```
df = pd.read_excel(
    'data.xlsx',
    sheet_name='Data',
    header=0,
    index_col=0,
    usecols='A:D',        # Excel column range
    nrows=100,
    skiprows=2,
    na_values=['N/A']
)
```

Installation note:

```
# Install openpyxl for Excel support
pip install openpyxl
```

Other File Formats

Pandas supports numerous other formats:

```
# JSON
df = pd.read_json('data.json')

# SQL databases
import sqlite3
conn = sqlite3.connect('database.db')
df = pd.read_sql_query('SELECT * FROM experiments', conn)

# HDF5 (for large datasets)
df = pd.read_hdf('data.h5', key='dataset')

# Parquet (efficient columnar storage)
df = pd.read_parquet('data.parquet')

# Stata, SPSS, SAS
df = pd.read_stata('data.dta')
df = pd.read_spss('data.sav')
df = pd.read_sas('data.sas7bdat')
```

---

4. Data Exploration

Initial Inspection

After loading data, always perform initial exploration:

Quick overview:

```
df.head()        # First 5 rows
df.head(10)      # First 10 rows
df.tail()        # Last 5 rows
df.sample(5)     # Random 5 rows
```

Data structure:

```
df.shape         # (n_rows, n_columns)
df.columns       # Column names
df.dtypes        # Data type of each column
df.info()        # Comprehensive overview: columns, types, non-null
counts, memory
```

Statistical summaries:

```
df.describe()    # Summary statistics for numerical columns
df.describe(include='all')  # Include non-numerical columns
```

The `describe()` output provides:

- `count` - Number of non-null values
- `mean` - Average value
- `std` - Standard deviation
- `min` - Minimum value
- `25%` - First quartile (25th percentile)
- `50%` - Median (50th percentile)
- `75%` - Third quartile (75th percentile)
- `max` - Maximum value

For categorical columns:

- `count` - Number of non-null values
- `unique` - Number of unique values
- `top` - Most frequent value
- `freq` - Frequency of top value

Column-Level Exploration

Unique values:

```
df['condition'].unique()           # Array of unique values
df['condition'].nunique()          # Count of unique values
df['condition'].value_counts()     # Frequency of each value
df['condition'].value_counts(normalize=True)  # Proportions
```

Missing data:

```
df.isnull().sum()                # Count missing values per column
df.isnull().sum() / len(df)      # Proportion missing per column
df.isnull().any()                # Which columns have any missing
values
df.isnull().all()                # Which columns are entirely missing
```

Data types:

```
df.dtypes                          # Data type of each column
df.select_dtypes(include='number')  # Select only numeric columns
df.select_dtypes(include=['object', 'string'])  # Select only text
columns
df.select_dtypes(exclude='number')  # Exclude numeric columns
```

Quick Visualizations

Pandas integrates with matplotlib for quick exploratory plots:

```
# Histogram of a column
df['reaction_time'].hist(bins=30)

# Distribution of categorical variable
df['condition'].value_counts().plot(kind='bar')

# Box plot for outlier detection
df.boxplot(column='reaction_time', by='condition')

# Scatter plot
df.plot.scatter(x='age', y='reaction_time')

# Correlation heatmap (requires seaborn)
import seaborn as sns
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
```

---

5. Data Selection and Indexing

Basic Column Selection

Selecting single columns:

```
# Returns a Series
df['reaction_time']
df.reaction_time  # Dot notation (only for valid Python identifiers)
```

Selecting multiple columns:

```
# Returns a DataFrame
df[['subject_id', 'reaction_time']]
df[['subject_id', 'reaction_time', 'accuracy']]
```

Boolean Indexing (Filtering Rows)

Boolean indexing allows you to filter rows based on conditions:

Simple conditions:

```
        # Single condition
        df[df['accuracy'] > 0.9]
        df[df['condition'] == 'drug']
        df[df['reaction_time'] >= 300]
```

Multiple conditions:

```
        # AND condition (both must be True)
        df[(df['accuracy'] > 0.9) & (df['reaction_time'] < 300)]

        # OR condition (at least one must be True)
        df[(df['condition'] == 'drug') | (df['condition'] == 'placebo')]

        # NOT condition
        df[~(df['accuracy'] < 0.8)]  # Same as df[df['accuracy'] >= 0.8]
```

Important: Use & for AND, | for OR, ~ for NOT. Regular Python `and`, `or`, `not` won't work with pandas boolean indexing.

Checking membership:

```
        # Using isin() for multiple values
        conditions = ['drug', 'placebo', 'control']
        df[df['condition'].isin(conditions)]

        # String contains
        df[df['gene_name'].str.contains('GRIN')]

        # Between values
        df[df['age'].between(20, 30)]
```

.loc[] - Label-Based Selection

`.loc[]` selects data by label (index and column names):

Syntax: `df.loc[row_labels, column_labels]`

```
        # Select specific rows by index label
        df.loc[0]        # Single row (returns Series)
        df.loc[[0, 1, 5]]  # Multiple rows (returns DataFrame)

        # Select specific columns
        df.loc[:, 'reaction_time']  # All rows, one column
        df.loc[:, ['subject_id', 'reaction_time']]  # All rows, multiple
columns

        # Select rows and columns
        df.loc[0:5, 'reaction_time']  # Rows 0-5, one column
        df.loc[0:5, ['reaction_time', 'accuracy']]  # Rows 0-5, multiple
columns

        # Boolean indexing with loc
```

```
df.loc[df['accuracy'] > 0.9, ['subject_id', 'reaction_time']]

# Assign values with loc
df.loc[df['accuracy'] > 0.95, 'high_performer'] = True
```

Important: With `.loc[]`, the end value is **included** in the slice. `df.loc[0:5]` returns rows 0, 1, 2, 3, 4, **and 5**.

.iloc[] - Position-Based Selection

`.iloc[]` selects data by integer position (like NumPy arrays):

Syntax: `df.iloc[row_positions, column_positions]`

```
# Select by row position
df.iloc[0]        # First row
df.iloc[-1]       # Last row
df.iloc[0:5]      # First 5 rows (0-4, end excluded)

# Select by column position
df.iloc[:, 0]     # All rows, first column
df.iloc[:, [0, 2, 3]]  # All rows, columns 0, 2, 3

# Select rows and columns by position
df.iloc[0:5, 0:3]  # First 5 rows, first 3 columns
df.iloc[[0, 5, 10], [1, 3]]  # Specific rows and columns

# Negative indexing
df.iloc[-5:]      # Last 5 rows
df.iloc[:, -1]    # Last column
```

Important: With `.iloc[]`, the end value is **excluded** from the slice (standard Python slicing). `df.iloc[0:5]` returns rows 0, 1, 2, 3, 4 (not 5).

When to Use .loc[] vs .iloc[]

Situation	Use
Select by label/name	`.loc[]`
Select by integer position	`.iloc[]`
Boolean indexing	`.loc[]` (preferred) or direct `df[condition]`
Working with non-numeric index	`.loc[]`
Need to assign values	`.loc[]` (safer)
Sequential positional access	`.iloc[]`

Advanced Indexing

Setting custom index:

```python
# Set a column as index
df.set_index('subject_id', inplace=True)

# Reset index to default integer index
df.reset_index(inplace=True)

# Multi-level index
df.set_index(['subject_id', 'session'], inplace=True)
```

Sorting:

```python
# Sort by values
df.sort_values('reaction_time')  # Ascending
df.sort_values('reaction_time', ascending=False)  # Descending
df.sort_values(['condition', 'reaction_time'])  # Multiple columns

# Sort by index
df.sort_index()
```

---

6. Data Cleaning

Handling Missing Data

Missing data is ubiquitous in real-world datasets. Pandas represents missing values as `NaN` (Not a Number) from NumPy.

Detecting Missing Data

```python
# Check for missing values
df.isnull()          # Boolean DataFrame (True where missing)
df.notnull()         # Inverse of isnull()
df.isnull().sum()    # Count missing values per column
df.isnull().sum().sum()  # Total missing values in DataFrame

# Check if any/all values are missing
df.isnull().any()    # True if column has any missing values
df.isnull().all()    # True if column is entirely missing

# Percentage of missing data
(df.isnull().sum() / len(df)) * 100

# Rows with any missing values
df[df.isnull().any(axis=1)]

# Rows with all values missing
df[df.isnull().all(axis=1)]
```

Removing Missing Data

```
        # Drop rows with any missing values
        df.dropna()

        # Drop rows where all values are missing
        df.dropna(how='all')

        # Drop rows with missing values in specific columns
        df.dropna(subset=['reaction_time', 'accuracy'])

        # Drop rows with fewer than N non-null values
        df.dropna(thresh=4)  # Keep rows with at least 4 non-null values

        # Drop columns with missing values
        df.dropna(axis=1)    # Drop any column with missing values
        df.dropna(axis=1, thresh=50)  # Keep columns with at least 50 non-null
values
```

Filling Missing Data

```
        # Fill with a constant value
        df.fillna(0)
        df['reaction_time'].fillna(df['reaction_time'].mean())

        # Fill with different values per column
        df.fillna({
            'reaction_time': df['reaction_time'].median(),
            'accuracy': 0.5,
            'condition': 'unknown'
        })

        # Forward fill (use previous value)
        df.ffill()

        # Backward fill (use next value)
        df.bfill()

        # Interpolate (for time series or ordered data)
        df['reaction_time'].interpolate(method='linear')
        df['reaction_time'].interpolate(method='polynomial', order=2)
        df['reaction_time'].interpolate(method='cubic')
```

Imputation Strategies

Different strategies for different data types:

1. **Numerical variables:**
    - Mean/median imputation (for normally distributed data)
    - Mode imputation (for skewed data)

- Forward/backward fill (for time series)
- Interpolation (for smooth temporal data)
- Regression imputation (predict from other variables)

2. **Categorical variables:**

   - Mode imputation (most frequent category)
   - Create "missing" category
   - Forward/backward fill (for temporal data)

3. **Time series:**

   - Forward fill (last observation carried forward)
   - Interpolation (linear, polynomial, spline)
   - Seasonal decomposition and filling

Example imputation workflow:

```
        # Identify missing data patterns
        missing_summary = df.isnull().sum()
        print(f"Missing data:\n{missing_summary[missing_summary > 0]}")

        # Numerical: impute with median (robust to outliers)
        df['reaction_time'] =
df['reaction_time'].fillna(df['reaction_time'].median())

        # Categorical: impute with mode
        df['condition'] = df['condition'].fillna(df['condition'].mode()[0])

        # Time series: interpolate
        df['neural_signal'] = df['neural_signal'].interpolate(method='linear')
```

Handling Outliers

Outliers can significantly affect analyses and should be detected and handled appropriately.

Detecting Outliers

Z-score method (assumes normal distribution):

```
        from scipy import stats

        # Calculate z-scores
        z_scores = np.abs(stats.zscore(df['reaction_time']))

        # Flag outliers (typically |z| > 3)
        outliers = z_scores > 3
        df[outliers]

        # Remove outliers
        df_no_outliers = df[z_scores <= 3]
```

IQR method (robust, doesn't assume normality):

```
        # Calculate IQR
        Q1 = df['reaction_time'].quantile(0.25)
        Q3 = df['reaction_time'].quantile(0.75)
        IQR = Q3 - Q1

        # Define outlier boundaries
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        # Flag outliers
        outliers = (df['reaction_time'] < lower_bound) | (df['reaction_time']
> upper_bound)

        # Remove outliers
        df_no_outliers = df[~outliers]
```

Handling Outliers

Strategies depend on the cause:

1. **Measurement errors** → Remove or correct
2. **Natural variation** → Keep, use robust statistics
3. **Biological signal** → Investigate separately
4. **Data entry errors** → Correct if possible

Options for handling:

```
        # Remove outliers
        df = df[~outliers]

        # Cap outliers (winsorization)
        df.loc[df['reaction_time'] > upper_bound, 'reaction_time'] =
upper_bound
        df.loc[df['reaction_time'] < lower_bound, 'reaction_time'] =
lower_bound

        # Transform data (reduce outlier influence)
        df['log_reaction_time'] = np.log(df['reaction_time'])

        # Flag for separate analysis
        df['is_outlier'] = outliers
```

Removing Duplicates

```
        # Find duplicates
        df.duplicated()  # Boolean Series (True for duplicates)
        df[df.duplicated()]  # Show duplicate rows
        df.duplicated().sum()  # Count duplicates
```

```
        # Remove duplicates
        df.drop_duplicates()

        # Keep first or last occurrence
        df.drop_duplicates(keep='first')   # Default
        df.drop_duplicates(keep='last')
        df.drop_duplicates(keep=False)     # Remove all duplicates

        # Check specific columns for duplicates
        df.drop_duplicates(subset=['subject_id', 'trial'])
```

---

7. Data Transformation

Data Type Conversions

Correct data types are essential for proper analysis:

```
        # Check current types
        df.dtypes

        # Convert to numeric
        df['subject_id'] = pd.to_numeric(df['subject_id'])
        df['age'] = pd.to_numeric(df['age'], errors='coerce')  # Invalid → NaN

        # Convert to integer (requires no missing values)
        df['trial'] = df['trial'].astype(int)

        # Convert to float
        df['reaction_time'] = df['reaction_time'].astype(float)

        # Convert to string
        df['subject_id'] = df['subject_id'].astype(str)

        # Convert to categorical (saves memory, enables categorical
operations)
        df['condition'] = df['condition'].astype('category')

        # Convert to datetime
        df['date'] = pd.to_datetime(df['date'])
        df['timestamp'] = pd.to_datetime(df['timestamp'], format='%Y-%m-%d
%H:%M:%S')
```

When to use categorical type:

- Fixed set of possible values (e.g., experimental conditions)
- Many repeated values (e.g., brain regions)
- Want to enforce valid values
- Memory efficiency for large datasets

Working with String Data

Pandas provides powerful string operations through the `.str` accessor:

```
        # Case conversion
        df['gene_name'].str.upper()
        df['gene_name'].str.lower()
        df['gene_name'].str.title()

        # String contains
        df[df['gene_name'].str.contains('GRIN')]
        df[df['brain_region'].str.contains('hippocampus', case=False)]

        # String matching
        df[df['gene_name'].str.startswith('GR')]
        df[df['gene_name'].str.endswith('2A')]
        df[df['gene_name'].str.match(r'GRIN[0-9]')]  # Regex

        # String length
        df['gene_name'].str.len()

        # String replace
        df['brain_region'].str.replace('_', ' ')
        df['condition'].str.replace(r'\d+', '', regex=True)  # Remove numbers

        # String split
        df['full_name'].str.split(' ')  # Returns list
        df[['first_name', 'last_name']] = df['full_name'].str.split(' ',
expand=True)

        # String strip (remove whitespace)
        df['gene_name'].str.strip()
        df['gene_name'].str.lstrip()  # Left only
        df['gene_name'].str.rstrip()  # Right only

        # Extract with regex
        df['subject_number'] = df['subject_id'].str.extract(r'(\d+)')
```

Creating New Columns

From existing columns:

```
        # Simple arithmetic
        df['rt_seconds'] = df['reaction_time'] / 1000

        # Multiple columns
        df['total_score'] = df['accuracy'] * df['speed_bonus']

        # Conditional creation
        df['performance'] = 'low'  # Default value
        df.loc[df['accuracy'] > 0.8, 'performance'] = 'high'
```

```python
# Using np.where() for if-else logic
df['correct'] = np.where(df['accuracy'] > 0.5, 'yes', 'no')

# Using np.select() for multiple conditions
conditions = [
    df['accuracy'] >= 0.9,
    df['accuracy'] >= 0.7,
    df['accuracy'] >= 0.5
]
choices = ['excellent', 'good', 'fair']
df['rating'] = np.select(conditions, choices, default='poor')
```

From functions:

```python
# Apply function to single column
def categorize_rt(rt):
    if rt < 250:
        return 'fast'
    elif rt < 400:
        return 'medium'
    else:
        return 'slow'

df['rt_category'] = df['reaction_time'].apply(categorize_rt)

# Apply function to multiple columns
def calculate_efficiency(row):
    return row['accuracy'] / (row['reaction_time'] / 1000)

df['efficiency'] = df.apply(calculate_efficiency, axis=1)

# Lambda functions for simple operations
df['log_rt'] = df['reaction_time'].apply(lambda x: np.log(x))
```

From binning:

```python
# Cut into bins with equal width
df['age_group'] = pd.cut(
    df['age'],
    bins=[0, 25, 35, 45, 100],
    labels=['18-25', '26-35', '36-45', '46+']
)

# Cut into quantiles (equal number per bin)
df['rt_quartile'] = pd.qcut(
    df['reaction_time'],
    q=4,
    labels=['Q1', 'Q2', 'Q3', 'Q4']
)
```

---

8. Grouping and Aggregation

Grouping and aggregation is one of pandas' most powerful features, implementing the "split-apply-combine" strategy:

1. **Split** - Divide data into groups based on criteria
2. **Apply** - Perform computations on each group independently
3. **Combine** - Merge results back into a single structure

Basic GroupBy Operations

```
# Group by single column
grouped = df.groupby('condition')

# Group by multiple columns
grouped = df.groupby(['condition', 'session'])

# Basic aggregations
grouped.mean()      # Mean of each group
grouped.sum()       # Sum of each group
grouped.count()     # Count non-null values
grouped.size()      # Count all values (including NaN)
grouped.std()       # Standard deviation
grouped.var()       # Variance
grouped.min()       # Minimum value
grouped.max()       # Maximum value
grouped.median()    # Median value
```

Example:

```
# Average reaction time by condition
df.groupby('condition')['reaction_time'].mean()

# Multiple statistics at once
df.groupby('condition')['reaction_time'].agg(['mean', 'std', 'count'])

# Group by multiple columns
df.groupby(['condition', 'session'])['accuracy'].mean()
```

Multiple Aggregations

Apply different functions to different columns:

```
df.groupby('condition').agg({
    'reaction_time': ['mean', 'std'],
    'accuracy': ['mean', 'median'],
    'subject_id': 'count'
})

# With custom function names
df.groupby('condition').agg({
```

```
        'reaction_time': [('mean_rt', 'mean'), ('sd_rt', 'std')],
        'accuracy': [('mean_acc', 'mean')]
    })
```

Custom aggregation functions:

```
    # Using named aggregation (pandas 0.25+)
    df.groupby('condition').agg(
        mean_rt=('reaction_time', 'mean'),
        sd_rt=('reaction_time', 'std'),
        n=('reaction_time', 'count'),
        median_acc=('accuracy', 'median')
    )

    # Custom function
    def coefficient_of_variation(x):
        return x.std() / x.mean()

    df.groupby('condition')['reaction_time'].agg(coefficient_of_variation)
```

Transform and Filter

Transform - Return a same-sized result (useful for normalization):

```
    # Z-score normalization within groups
    df['rt_zscore'] = df.groupby('condition')['reaction_time'].transform(
        lambda x: (x - x.mean()) / x.std()
    )

    # Percent of group mean
    df['rt_pct_of_mean'] = df.groupby('condition')
['reaction_time'].transform(
        lambda x: x / x.mean() * 100
    )

    # Rank within group
    df['rank_in_condition'] = df.groupby('condition')
['reaction_time'].rank()
```

Filter - Keep only groups that meet a condition:

```
    # Keep groups with more than 10 observations
    df.groupby('condition').filter(lambda x: len(x) > 10)

    # Keep groups where mean accuracy > 0.8
    df.groupby('subject_id').filter(lambda x: x['accuracy'].mean() > 0.8)
```

Iterating Over Groups

```
    # Iterate through groups
    for name, group in df.groupby('condition'):
```

```
    print(f"Condition: {name}")
    print(f"Size: {len(group)}")
    print(f"Mean RT: {group['reaction_time'].mean():.2f}\n")

# Get specific group
control_group = df.groupby('condition').get_group('control')
```

---

9. Reshaping and Combining Data

Pivot Tables

Pivot tables reorganize data for easier analysis:

```
# Basic pivot
pivot = df.pivot_table(
    values='reaction_time',
    index='subject_id',
    columns='condition',
    aggfunc='mean'
)

# Multiple value columns
pivot = df.pivot_table(
    values=['reaction_time', 'accuracy'],
    index='subject_id',
    columns='condition',
    aggfunc='mean'
)

# Multiple aggregation functions
pivot = df.pivot_table(
    values='reaction_time',
    index='subject_id',
    columns='condition',
    aggfunc=['mean', 'std', 'count']
)

# With margins (row and column totals)
pivot = df.pivot_table(
    values='reaction_time',
    index='subject_id',
    columns='condition',
    aggfunc='mean',
    margins=True
)
```

Melting (Wide to Long Format)

Convert wide format to long format (useful for plotting and statistical analysis):

```
        # Example wide format
        wide_df = pd.DataFrame({
            'subject': [1, 2, 3],
            'baseline': [10, 12, 11],
            'drug': [15, 18, 16],
            'washout': [11, 13, 12]
        })

        # Melt to long format
        long_df = wide_df.melt(
            id_vars=['subject'],
            value_vars=['baseline', 'drug', 'washout'],
            var_name='condition',
            value_name='response'
        )

        # Result:
        #    subject condition  response
        # 0        1  baseline        10
        # 1        2  baseline        12
        # 2        3  baseline        11
        # 3        1      drug        15
        # ...
```

Concatenating DataFrames

Combine DataFrames along rows or columns:

```
        # Concatenate along rows (stack vertically)
        df_combined = pd.concat([df1, df2, df3])

        # Reset index after concatenation
        df_combined = pd.concat([df1, df2, df3], ignore_index=True)

        # Concatenate along columns (side by side)
        df_combined = pd.concat([df1, df2], axis=1)

        # Handle different column names
        df_combined = pd.concat([df1, df2], join='inner')  # Intersection of
columns
        df_combined = pd.concat([df1, df2], join='outer')  # Union of columns
(default)

        # Add identifier for source DataFrame
        df_combined = pd.concat(
            [df1, df2, df3],
```

```
    keys=['exp1', 'exp2', 'exp3']
)
```

Merging DataFrames

Merge DataFrames based on common columns (like SQL joins):

```
# Inner join (only matching rows)
merged = pd.merge(df1, df2, on='subject_id', how='inner')

# Left join (all rows from left, matching from right)
merged = pd.merge(df1, df2, on='subject_id', how='left')

# Right join (all rows from right, matching from left)
merged = pd.merge(df1, df2, on='subject_id', how='right')

# Outer join (all rows from both)
merged = pd.merge(df1, df2, on='subject_id', how='outer')

# Merge on multiple columns
merged = pd.merge(df1, df2, on=['subject_id', 'session'])

# Merge when column names differ
merged = pd.merge(
    df1, df2,
    left_on='subject_id',
    right_on='participant_id'
)

# Merge on index
merged = pd.merge(df1, df2, left_index=True, right_index=True)
```

Merge types:

Type	SQL Equivalent	Description
inner	INNER JOIN	Only rows present in both
left	LEFT OUTER JOIN	All rows from left, matching from right
right	RIGHT OUTER JOIN	All rows from right, matching from left
outer	FULL OUTER JOIN	All rows from both
cross	CROSS JOIN	Cartesian product

---

10. Visualization with Pandas

Pandas provides built-in plotting capabilities using matplotlib:

Basic Plots

```
import matplotlib.pyplot as plt

# Line plot
df['reaction_time'].plot()
plt.show()

# Multiple columns
df[['reaction_time', 'accuracy']].plot()

# Histogram
df['reaction_time'].hist(bins=30)

# Box plot
df.boxplot(column='reaction_time', by='condition')

# Scatter plot
df.plot.scatter(x='age', y='reaction_time')

# Bar plot
df['condition'].value_counts().plot(kind='bar')
```

Customization

```
# Customize plot
df['reaction_time'].plot(
    kind='hist',
    bins=30,
    title='Distribution of Reaction Times',
    xlabel='Reaction Time (ms)',
    ylabel='Frequency',
    color='steelblue',
    alpha=0.7,
    figsize=(10, 6)
)

# Save plot
plt.savefig('rt_distribution.png', dpi=300, bbox_inches='tight')
```

Plot Types

```
# Line plot
df.plot(x='trial', y='reaction_time', kind='line')

# Bar plot
df.groupby('condition')['reaction_time'].mean().plot(kind='bar')

# Horizontal bar
df.groupby('condition')['reaction_time'].mean().plot(kind='barh')
```

```
        # Histogram
        df['reaction_time'].plot(kind='hist', bins=20)

        # Box plot
        df.boxplot(column='reaction_time', by='condition')

        # Area plot
        df.plot(kind='area', x='time', y='neural_activity')

        # Pie chart
        df['condition'].value_counts().plot(kind='pie')

        # Scatter plot with size and color
        df.plot.scatter(
            x='age',
            y='reaction_time',
            s=df['accuracy']*100,  # Size
            c=df['session'],        # Color
            cmap='viridis',
            alpha=0.5
        )
```

Subplots

```
        # Multiple subplots
        fig, axes = plt.subplots(2, 2, figsize=(12, 10))

        df['reaction_time'].hist(ax=axes[0, 0], bins=30)
        df.boxplot(column='reaction_time', by='condition', ax=axes[0, 1])
        df.plot.scatter(x='age', y='reaction_time', ax=axes[1, 0])
        df['condition'].value_counts().plot(kind='bar', ax=axes[1, 1])

        plt.tight_layout()
        plt.show()
```

Note: Next lecture covers advanced visualization with Seaborn and Plotly!

---

11. Working with Datetime Data

Timeseries Basics

```
        # Example data: one value per day (replace with your own recordings)
        df = pd.DataFrame({
            'date': pd.date_range('2000-01-01', periods=120,
freq='D').astype(str),
            'reaction_time': np.random.normal(350, 40, 120),
        })
```

```
        # Create datetime index
        df['date'] = pd.to_datetime(df['date'])
        df = df.set_index('date')

        # Datetime components
        df['year'] = df.index.year
        df['month'] = df.index.month
        df['day'] = df.index.day
        df['dayofweek'] = df.index.dayofweek

        # Time-based selection (use .loc with partial date strings)
        df.loc['2000-01']                 # All data from January 2000
        df.loc['2000-01':'2000-03']       # January to March 2000
        df.loc['2000-01-15']              # One specific day

        # Resampling (change frequency); numeric columns only
        df[['reaction_time']].resample('D').mean()   # Daily mean
        df[['reaction_time']].resample('W').sum()    # Weekly sum
        df[['reaction_time']].resample('ME').count() # Monthly count ('M' was
removed in pandas 3)

        # Rolling windows
        df['reaction_time'].rolling(window=7).mean()  # 7-day moving average
        df['reaction_time'].rolling(window=7).std()   # 7-day rolling std

        # Time shifts
        df['reaction_time'].shift(1)   # Lag by 1 period
        df['reaction_time'].shift(-1)  # Lead by 1 period
        df['reaction_time'].diff()     # First difference
        df['reaction_time'].pct_change()  # Percentage change
```

---

12. Signal Processing Basics

Filtering Signals

```
        from scipy import signal

        # Low-pass filter (remove high frequencies)
        b, a = signal.butter(N=4, Wn=0.1, btype='low')
        filtered_signal = signal.filtfilt(b, a, neural_signal)

        # High-pass filter (remove low frequencies/drift)
        b, a = signal.butter(N=4, Wn=0.01, btype='high')
        filtered_signal = signal.filtfilt(b, a, neural_signal)

        # Band-pass filter (keep specific frequency range)
        b, a = signal.butter(N=4, Wn=[0.05, 0.2], btype='band')
        filtered_signal = signal.filtfilt(b, a, neural_signal)
```

```
# Band-stop filter (remove specific frequency range)
b, a = signal.butter(N=4, Wn=[0.05, 0.15], btype='bandstop')
filtered_signal = signal.filtfilt(b, a, neural_signal)
```

Filter types:

- **Butterworth** - Smooth frequency response
- **Chebyshev** - Steeper roll-off, some ripple
- **Bessel** - Best phase response
- **Elliptic** - Steepest roll-off

Smoothing

```
# Moving average
window_size = 10
smoothed = df['neural_signal'].rolling(window=window_size).mean()

# Savitzky-Golay filter (polynomial smoothing)
from scipy.signal import savgol_filter
smoothed = savgol_filter(neural_signal, window_length=11, polyorder=3)

# Gaussian filter
from scipy.ndimage import gaussian_filter1d
smoothed = gaussian_filter1d(neural_signal, sigma=2)
```

Fourier Analysis

```
# FFT (Fast Fourier Transform)
from scipy.fft import fft, fftfreq

# Compute FFT
fft_values = fft(neural_signal)
freqs = fftfreq(len(neural_signal), d=1/sampling_rate)

# Power spectrum
power = np.abs(fft_values)**2

# Plot frequency domain
plt.plot(freqs[:len(freqs)//2], power[:len(power)//2])
plt.xlabel('Frequency (Hz)')
plt.ylabel('Power')
```

---

13. The SciPy Library

SciPy builds on NumPy to provide scientific computing capabilities:

Main Modules

- **scipy.stats** - Statistical functions and distributions
- **scipy.signal** - Signal processing
- **scipy.optimize** - Optimization algorithms
- **scipy.integrate** - Integration routines
- **scipy.interpolate** - Interpolation
- **scipy.linalg** - Linear algebra
- **scipy.ndimage** - Image processing
- **scipy.spatial** - Spatial algorithms
- **scipy.cluster** - Clustering algorithms

Installation

```
pip install scipy
```

Import Convention

```
from scipy import stats, signal, optimize
import scipy as sp
```

---

14. Statistical Testing

Common Statistical Tests

T-Tests

Independent samples t-test (compare two independent groups):

```
from scipy import stats

# Example: Compare reaction times between two conditions
control = df[df['condition'] == 'control']['reaction_time']
drug = df[df['condition'] == 'drug']['reaction_time']

# Perform t-test
t_stat, p_value = stats.ttest_ind(control, drug)

print(f"t-statistic: {t_stat:.4f}")
print(f"p-value: {p_value:.4f}")

if p_value < 0.05:
    print("Significant difference between groups")
else:
    print("No significant difference")
```

Paired samples t-test (compare two related measurements):

```
# Example: Before and after treatment
before = df['baseline_score']
after = df['treatment_score']

t_stat, p_value = stats.ttest_rel(before, after)
```

One-sample t-test (compare sample mean to hypothesized value):

```
# Test if mean reaction time differs from 300ms
t_stat, p_value = stats.ttest_1samp(df['reaction_time'], 300)
```

Assumptions:

- Continuous data
- Independent observations (except paired t-test)
- Approximately normal distribution
- Homogeneity of variance (for independent t-test)

Checking assumptions:

```
# Normality test (Shapiro-Wilk)
stat, p = stats.shapiro(control)
if p > 0.05:
    print("Data appears normally distributed")

# Levene's test for equal variances
stat, p = stats.levene(control, drug)
if p > 0.05:
    print("Variances are equal")
    # Use standard t-test
else:
    print("Variances are unequal")
    # Use Welch's t-test
    t_stat, p_value = stats.ttest_ind(control, drug, equal_var=False)
```

Non-Parametric Tests

Use when data doesn't meet t-test assumptions:

Mann-Whitney U test (independent samples):

```
# Non-parametric alternative to independent t-test
u_stat, p_value = stats.mannwhitneyu(control, drug)
```

Wilcoxon signed-rank test (paired samples):

```
# Non-parametric alternative to paired t-test
w_stat, p_value = stats.wilcoxon(before, after)
```

Kruskal-Wallis H test (more than two groups):

```
# Non-parametric alternative to one-way ANOVA
h_stat, p_value = stats.kruskal(group1, group2, group3)
```

ANOVA

One-way ANOVA (compare means of three or more groups):

```
# Example: Compare reaction times across multiple conditions
control = df[df['condition'] == 'control']['reaction_time']
drug_a = df[df['condition'] == 'drug_a']['reaction_time']
drug_b = df[df['condition'] == 'drug_b']['reaction_time']

f_stat, p_value = stats.f_oneway(control, drug_a, drug_b)

print(f"F-statistic: {f_stat:.4f}")
print(f"p-value: {p_value:.4f}")
```

Post-hoc tests (if ANOVA is significant):

```
# Tukey's HSD test requires statsmodels
from statsmodels.stats.multicomp import pairwise_tukeyhsd

tukey = pairwise_tukeyhsd(
    df['reaction_time'],
    df['condition'],
    alpha=0.05
)
print(tukey)
```

Chi-Square Test

Chi-square test of independence (categorical variables):

```
# Example: Test if condition and response are independent
contingency_table = pd.crosstab(df['condition'], df['response'])

chi2, p_value, dof, expected =
stats.chi2_contingency(contingency_table)

print(f"Chi-square statistic: {chi2:.4f}")
print(f"p-value: {p_value:.4f}")
print(f"Degrees of freedom: {dof}")
```

Effect Sizes

P-values tell us if an effect exists; effect sizes tell us how large it is:

Cohen's d (standardized mean difference):

```
def cohens_d(group1, group2):
    n1, n2 = len(group1), len(group2)
    var1, var2 = group1.var(), group2.var()
    pooled_std = np.sqrt(((n1-1)*var1 + (n2-1)*var2) / (n1+n2-2))
    return (group1.mean() - group2.mean()) / pooled_std
```

```
d = cohens_d(control, drug)
print(f"Cohen's d: {d:.4f}")

# Interpretation:
# |d| < 0.2: small effect
# |d| ~ 0.5: medium effect
# |d| > 0.8: large effect
```

---

15. Distribution Fitting

Understanding Distributions

Many biological and neural processes follow known probability distributions. Fitting distributions helps us:

- Understand data-generating processes
- Make predictions about unobserved data
- Choose appropriate statistical tests
- Identify outliers

Common Distributions in Neuroscience

Distribution	Common Applications
Normal	Measurement errors, many biological variables
Exponential	Inter-spike intervals, waiting times
Poisson	Spike counts in fixed time windows
Log-normal	Synaptic weights, reaction times
Gamma	Reaction times, neural response latencies
Beta	Proportions, probabilities

Fitting Distributions with SciPy

```
from scipy import stats
import numpy as np
import matplotlib.pyplot as plt

# Generate or load data
# Example: Inter-spike intervals (ISI)
isi_data = df['interspike_interval'].dropna()

# Fit exponential distribution
params = stats.expon.fit(isi_data)
loc, scale = params  # loc = offset, scale = 1/lambda

print(f"Fitted parameters: loc={loc:.4f}, scale={scale:.4f}")
```

```
        # Generate theoretical PDF
        x = np.linspace(isi_data.min(), isi_data.max(), 1000)
        pdf = stats.expon.pdf(x, loc=loc, scale=scale)

        # Visualize fit
        plt.figure(figsize=(10, 6))
        plt.hist(isi_data, bins=50, density=True, alpha=0.6, label='Data')
        plt.plot(x, pdf, 'r-', linewidth=2, label='Fitted exponential')
        plt.xlabel('Inter-spike interval (ms)')
        plt.ylabel('Probability density')
        plt.legend()
        plt.title('Exponential Distribution Fit to ISI Data')
        plt.show()
```

Fitting Different Distributions

```
        # Normal distribution
        mu, sigma = stats.norm.fit(reaction_times)
        pdf_normal = stats.norm.pdf(x, mu, sigma)

        # Log-normal distribution
        shape, loc, scale = stats.lognorm.fit(reaction_times)
        pdf_lognorm = stats.lognorm.pdf(x, shape, loc, scale)

        # Gamma distribution
        shape, loc, scale = stats.gamma.fit(reaction_times)
        pdf_gamma = stats.gamma.pdf(x, shape, loc, scale)

        # Compare fits visually
        plt.figure(figsize=(12, 6))
        plt.hist(reaction_times, bins=30, density=True, alpha=0.6,
label='Data')
        plt.plot(x, pdf_normal, label='Normal')
        plt.plot(x, pdf_lognorm, label='Log-normal')
        plt.plot(x, pdf_gamma, label='Gamma')
        plt.legend()
        plt.show()
```

Goodness of Fit Tests

Kolmogorov-Smirnov Test

Tests if data comes from a specific distribution:

```
        # KS test for normal distribution
        ks_stat, p_value = stats.kstest(reaction_times, stats.norm(mu,
sigma).cdf)

        print(f"KS statistic: {ks_stat:.4f}")
        print(f"p-value: {p_value:.4f}")
```

```
if p_value > 0.05:
    print("Data consistent with normal distribution")
else:
    print("Data significantly different from normal distribution")
```

Chi-Square Goodness of Fit

```
# Create histogram bins
observed, bin_edges = np.histogram(reaction_times, bins=20)

# Calculate expected frequencies from fitted distribution
bin_centers = (bin_edges[:-1] + bin_edges[1:]) / 2
expected = len(reaction_times) * np.diff(
    stats.norm.cdf(bin_edges, mu, sigma)
)

# Chi-square test
chi2, p_value = stats.chisquare(observed, expected)

print(f"Chi-square statistic: {chi2:.4f}")
print(f"p-value: {p_value:.4f}")
```

Q-Q Plot

Quantile-quantile plots visually assess if data follows a theoretical distribution:

```
from scipy import stats
import matplotlib.pyplot as plt

# Q-Q plot for normal distribution
stats.probplot(reaction_times, dist="norm", plot=plt)
plt.title("Q-Q Plot: Reaction Times vs Normal Distribution")
plt.show()

# Interpretation:
# Points on the line → data matches distribution
# Systematic deviations → data doesn't match distribution
```

---

16. Correlation Analysis

Pearson Correlation

Pearson correlation measures **linear** relationships between continuous variables.

Correlation coefficient (r):

- Range: -1 to +1
- r = 1: Perfect positive linear relationship

- r = 0: No linear relationship
- r = -1: Perfect negative linear relationship

Formula:

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

Assumptions:

- Linear relationship
- Continuous variables
- Bivariate normal distribution
- No outliers

Computing Pearson correlation:

```
from scipy import stats

# Single pair of variables
r, p_value = stats.pearsonr(df['age'], df['reaction_time'])

print(f"Pearson r: {r:.4f}")
print(f"p-value: {p_value:.4f}")

# Interpretation of r:
if abs(r) < 0.3:
    strength = "weak"
elif abs(r) < 0.7:
    strength = "moderate"
else:
    strength = "strong"

direction = "positive" if r > 0 else "negative"
print(f"{strength} {direction} correlation")
```

Correlation matrix:

```
# Compute correlation matrix for all numeric columns
corr_matrix = df[['age', 'reaction_time', 'accuracy', 'iq']].corr()

print(corr_matrix)

# Visualize with heatmap
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', center=0,
            vmin=-1, vmax=1, square=True, linewidths=1)
plt.title('Correlation Matrix')
plt.show()
```

Finding highly correlated pairs:

```
        # Get correlation pairs
        corr_pairs = corr_matrix.unstack()

        # Filter for high correlations (exclude diagonal)
        high_corr = corr_pairs[
            (abs(corr_pairs) > 0.7) & (abs(corr_pairs) < 1.0)
        ].sort_values(ascending=False)

        print(high_corr)
```

Important caveats:

1. **Correlation ≠ causation** - Strong correlation doesn't imply causal relationship
2. **Linear only** - Pearson only captures linear relationships
3. **Outliers** - Sensitive to extreme values
4. **Sample size** - p-values depend on n; small effects can be "significant" with large n

Spearman Correlation

Spearman's rank correlation measures **monotonic** (not necessarily linear) relationships.

When to use Spearman instead of Pearson:

- Non-linear but monotonic relationships
- Ordinal data (e.g., Likert scales)
- Non-normal distributions
- Presence of outliers

Spearman's ρ (rho):

- Pearson correlation computed on **ranks** instead of raw values
- Range: -1 to +1
- Robust to outliers
- Can detect non-linear monotonic relationships

```
        # Spearman correlation
        rho, p_value = stats.spearmanr(df['age'], df['reaction_time'])

        print(f"Spearman ρ: {rho:.4f}")
        print(f"p-value: {p_value:.4f}")

        # Correlation matrix with Spearman
        spearman_matrix = df[['age', 'reaction_time',
'accuracy']].corr(method='spearman')
```

Kendall's Tau

Kendall's τ (tau) is another rank-based correlation measure.

Advantages of Kendall's τ:

- More robust than Spearman for small samples
- Better handles tied ranks
- Direct probabilistic interpretation

When to use Kendall instead of Spearman:

- Small sample size (n < 30)
- Many tied ranks
- Need more robust measure

```
# Kendall's tau
tau, p_value = stats.kendalltau(df['age'], df['reaction_time'])

print(f"Kendall's τ: {tau:.4f}")
print(f"p-value: {p_value:.4f}")
```

Choosing the Right Correlation Method

Situation	Method
Linear relationship, normal data	Pearson
Monotonic relationship	Spearman
Ordinal data	Spearman or Kendall
Non-normal distributions	Spearman or Kendall
Outliers present	Spearman or Kendall
Small sample with ties	Kendall

Neuroscience Applications

Gene expression analysis:

```
# Correlation between gene expressions
gene1 = df['BDNF_expression']
gene2 = df['GRIN2A_expression']
r, p = stats.pearsonr(gene1, gene2)
```

Brain-behavior relationships:

```
# Correlation between neural activity and behavior
neural_activity = df['hippocampus_activation']
memory_score = df['memory_performance']
r, p = stats.pearsonr(neural_activity, memory_score)
```

Dose-response curves:

```
# Spearman for non-linear dose-response
rho, p = stats.spearmanr(df['drug_dose'], df['response'])
```

---

17. Best Practices and Workflow

The Tidy Data Workflow

1. **Start with raw data** - Never modify original files
2. **Document everything** - Use comments and markdown cells
3. **Check data quality** - Inspect, visualize, identify issues
4. **Clean systematically** - Handle missing values, outliers, errors
5. **Transform as needed** - Create derived variables
6. **Analyze** - Statistical tests, modeling
7. **Visualize findings** - Communicate results clearly
8. **Validate results** - Check assumptions, cross-validate

Code Organization

```
# 1. Imports
import pandas as pd
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt
import seaborn as sns

# 2. Configuration
pd.set_option('display.max_columns', 50)
plt.style.use('seaborn-v0_8-darkgrid')
np.random.seed(42)

# 3. Load data
df = pd.read_csv('data.csv')

# 4. Initial exploration
print(df.head())
print(df.info())
print(df.describe())

# 5. Data cleaning
df = df.dropna(subset=['critical_column'])
df['date'] = pd.to_datetime(df['date'])

# 6. Analysis
results = df.groupby('condition')['response'].mean()

# 7. Visualization
fig, ax = plt.subplots(figsize=(10, 6))
results.plot(kind='bar', ax=ax)
plt.show()
```

Common Pitfalls to Avoid

1. **Modifying original data** - Always work on copies

2. **Not checking assumptions** - Validate test requirements
3. **P-hacking** - Don't test multiple hypotheses without correction
4. **Ignoring missing data patterns** - Missing data may not be random
5. **Forgetting to set random seeds** - Results won't be reproducible
6. **Over-interpreting correlations** - Remember: correlation ≠ causation

Performance Tips

```python
# Use vectorized operations (fast)
df['new_col'] = df['col1'] * df['col2']

# Avoid loops (slow)
for i in range(len(df)):
    df.loc[i, 'new_col'] = df.loc[i, 'col1'] * df.loc[i, 'col2']

# Use categorical type for repeated strings
df['condition'] = df['condition'].astype('category')

# Read only needed columns
df = pd.read_csv('data.csv', usecols=['col1', 'col2', 'col3'])

# Use chunks for large files
for chunk in pd.read_csv('large_file.csv', chunksize=10000):
    process(chunk)
```

Reproducibility Checklist

- [ ] Set random seeds (`np.random.seed(42)`)
- [ ] Document package versions
- [ ] Save processed data with timestamps
- [ ] Version control with Git
- [ ] Include environment specifications (requirements.txt)
- [ ] Clear markdown documentation in notebooks
- [ ] Separate data preparation from analysis
- [ ] Save intermediate results

Summary

You've now covered the essential tools for data manipulation and statistical analysis in Python:

Pandas:

- Data structures (Series, DataFrame)
- Reading and writing data
- Data exploration and selection
- Data cleaning and transformation
- Grouping and aggregation
- Reshaping and combining
- Datetime handling

- Built-in visualization

SciPy:

- Statistical hypothesis testing
- Distribution fitting and goodness of fit
- Correlation analysis
- Signal processing basics

Next Steps:

- Practice with real neuroscience datasets
- Explore advanced visualization (next lecture!)
- Learn about machine learning workflows
- Dive deeper into time series analysis
- Study advanced statistical methods

Additional Resources:

- Pandas documentation: https://pandas.pydata.org/docs/
- SciPy documentation: https://docs.scipy.org/doc/scipy/
- Wes McKinney's "Python for Data Analysis" (2nd edition)
- Jake VanderPlas's "Python Data Science Handbook"

This handout is part of the "Introduction to Scientific Programming" course at CNC-UC, University of Coimbra. For questions or clarifications, please contact the course instructor. **Document Version**: 1.0
License: CC BY 4.0